“Title Case -- Digital Library Management System (DBMS) - A SQL-Based Solution”

A Technical Report submitted in partial fulfillment of the requirements for the degree of

Bachelor Of Technology

Project Leader- [Aviral Verma] [202310101110050]
Team Member- [Udit Raj Pandey] [202310101110007]

Supervisor

Mrs. Neeta Bhusal Sharma

(Department of computer Science & Engineering)

“ Institute Name”

SHRI RAMSWAROOP MEMORIAL UNIVERSITY

BARABANKI, UTTAR PRADESH -225003

« 2025»

Abstract :-

The Library Management System (LMS) project aims to digitalize and optimize essential library operations, eliminating the challenges of traditional paper-based management. This system uses a well-structured relational SQL database to organize and control the primary components of a library. Built upon normalized relational tables—Members, Books, and Transactions—the database ensures accuracy through strict data types, primary keys, foreign keys, and relationship constraints. These mechanisms guarantee data consistency by restricting invalid entries, such as issuing books to non-existent members or referencing unavailable inventory. The system efficiently maintains member profiles, tracks membership activity, and manages book records by clearly distinguishing between total and currently available copies. The Transactions table captures every borrowing and return event, forming the operational backbone of the system. To enhance automation, fine calculation for overdue returns is handled through an SQL function, minimizing manual effort. Key operations such as book issuing and returning are implemented as stored procedures, ensuring secure, reusable, and modular business logic. Additionally, analytical views like Library Stats and Current Issued Books provide simplified and performance-optimized access to aggregated information for reporting and monitoring. Altogether, this SQL-driven architecture delivers a reliable, scalable, and efficient backend that can support a fully functional library management application.

Keywords: Library Management System, SQL Database, Members, Books, Transactions, Issued Books, Fine Calculation, Library Analytics

Table of Contents

1. **Objective/Aim**
   - **1.1 Problem Statement**
   - **1.2 Project Objectives**
   - **1.3 Scope and Limitations**
2. **Technical Details**
   - **2.1 System Analysis and Design**
   - **2.2 Entity -Relationship(ER) Model**
   - **2.3 DAtebase Schema**
3. **System Implementation**
   - **3.1 Database and Table Creation**
   - **3.2 Stored Procedures and Functions**
   - **3.3 Views**
   - **3.4 Sample Data**
4. **Testing and Demonstration**
   - **4.1 Test Case 1: View Available Books**
   - **4.2 Test Case 2: View Overdue Books**
   - **4.3 Test Case 3: Library Statistics**
   - **4.4 Test Case 4: Members with Fines**
   - **4.5 Test Case 5: Issuing a Book (Procedure)**
   - **4.6 Test Case 6: Calculating a Member's Fine (Function)**
5. **Innovativeness & Usefulness**
6. **Current Status of Development**
7. **Market Potential & Competitive Advantage**
8. **Conclusion and Future Scope**
   - **8.1 Conclusion**
   - **8.2 Future Scope**
9. **Appendix: Complete SQL Code**

10. **Reference**

1. Objective / Aim

1.1. Problem Statement

Traditional libraries often rely on manual, paper-based systems or outdated spreadsheets for managing their inventory and member activities. This approach is inefficient and suffers from several problems:

1.1.1 **Error-Prone**: Manual data entry leads to spelling mistakes, incorrect tracking of due dates, and lost records.

1.1.2 **Time-Consuming:** Tasks such as checking a book's availability, identifying overdue books, and calculating fines are laborious for library staff.

1.1.3 **Poor Reporting:** Generating statistics on popular books, member activity, or total fines is difficult and often inaccurate.

1.1.4 Scalability Issues: As the collection of books and the number of members grow, a manual system becomes unmanageable.

1.2. Project Objectives

This project aims to design, develop, and implement a relational database backend for a Library Management System to resolve these issues.

The primary objectives are:

1.2.1 To create a centralized database to store all information related to books, members, and transactions.

1.2.2 To automate core library processes, including issuing books, returning books, and tracking due dates.

1.2.3 To ensure data integrity and accuracy through the use of primary keys, foreign keys, and constraints**.**

1.2.4 To implement business logic for automatically calculating fines for overdue books.

1.2.5 To provide easy access to information and reporting through a set of predefined SQL queries and views.

1.2.6 To build a scalable foundation that can support future development, such as a web-based user interface.

1.3. Scope and Limitations

Scope:

1.3.1 **Member Management:** The system can add new members, update their information, and track their status (Active/Inactive).

1.3.2 **Book Management**: The system catalogs books with details like title, author, and ISBN. It also tracks the total number of copies and the number of copies currently available.

1.3.3 **Transaction Management:** The system logs every book issuance and return, including dates and fine amounts.

1.3.4 **Reporting**: The system includes views to see currently issued books and overall library statistics.

1.3.5 Limitations:

1.3.6 **Backend Only:** This project delivers only the SQL database backend. It does not include a graphical user interface (GUI) for end-users (e.g., librarians or members). All interactions are performed via SQL commands.

1.3.7 No User Authentication: The system does not have roles or permissions (e.g., Librarian vs. Member).

1.3.8 **No Reservation System:** The current schema does not support reserving a book that is currently checked out.

2. **Technical Details**

2.1. System Analysis and Design

The system is designed as a relational database using a standard client-server architecture. The SQL database (e.g., MySQL, PostgreSQL) acts as the server, storing and managing all data. A database administrator or a future application (the client) interacts with the database by executing SQL queries and stored procedures.

2.2. Entity-Relationship (ER) Model

The database design is based on three primary entities:

2.2.1 Members: Represents the library patrons.

- **Attributes:** member_id (Primary Key), Full_name, Email, Phone, Membership_date, status.

2.2.2 Books: Represents the books in the library's collection.

2.2.3 Attributes: book_id (Primary Key), title, author, genre, published_year, isbn, total_copies, available_copies.

2.2.4 Transactions: An associative entity that links Members and Books, representing the act of a member borrowing a book.

2.2.5 Attributes: transaction_id (Primary Key), book_id (Foreign Key), member_id (Foreign Key), issue_date, due_date, return_date, fine_amount, status.

Relationships:

1 A **one-to-many** relationship exists between a Member and Transactions (one member can have many transactions).

2 A **one-to-many** relationship exists between a Book and Transactions (one book can be involved in many transactions).

3 This structure effectively models the **many-to-many** relationship between Members and Books (a member can borrow many books, and a book can be borrowed by many members over time).

2.3. Database Schema

Based on the ER model, the following three tables are defined:

1.2.1 Members Table: Stores information about each library member.

- Member_id: INT, AUTO_INCREMENT, PRIMARY KEY
- Full_name: VARCHAR(100), NOT NULL
- Gmail: VARCHAR(100), UNIQUE
- Phone: VARCHAR(15)
- Membership_date: DATE, NOT NULL
- Status: ENUM('Active', 'Inactive')
- Created_at: TIMESTAMP

2.3.2 Books Table: Stores information about each book in the catalog.

- Book_id: INT, AUTO_INCREMENT, PRIMARY KEY
- Title: VARCHAR(255), NOT NULL
- Author: VARCHAR(100), NOT NULL
- Genre: VARCHAR(50)
- Published_year: YEAR
- Isbn: VARCHAR(20), UNIQUE
- Total_copies: INT
- available_copies: INT (This is denormalized for performance, allowing quick checks on availability).

3. Transactions Table: A record of every book loan.

- Transaction_id: INT, AUTO_INCREMENT, PRIMARY KEY
- Book_id: INT, NOT NULL, FOREIGN KEY (references Books.book_id)
- Member_id: INT, NOT NULL, FOREIGN KEY (references Members.member_id)
- Issue_date: DATE, NOT NULL
- Due_date: DATE, NOT NULL (Typically 14 days from issue_date)
- Return_date: DATE, NULL (NULL indicates the book is still issued)
- Fine_amount: DECIMAL(8,2)
- Status: ENUM('Issued', 'Returned', 'Overdue')
- reated at: TIMESTAMP

3. System Implementation

The system was implemented using SQL. The full code is available in the Appendix, but key components are explained here.

3.1. Database and Table Creation

The database library_management_system is created first. The CREATE TABLE statements translate the schema design (Section 2.3) into physical database structures, enforcing data types and constraints like NOT NULL, UNIQUE, PRIMARY KEY, and FOREIGN KEY. The foreign key constraints are crucial as they enforce referential integrity, preventing "orphan" records (e.g., a transaction for a non-existent member).

3.2. Stored Procedures and Functions

To encapsulate business logic and improve security, stored procedures and functions are used.

3.1.1 **IssueBook(p_member_id, p_book_id, p_issue_date) (Procedure):**

3.1.2 **Purpose:** Automates the process of issuing a book.

3.1.3 **Logic:**

3.1.3.1 It first checks the available_copies of the requested p_book_id.

3.1.3.2 If available_copies > 0, it inserts a new record into the Transactions table, setting the due_date to 14 days after the p_issue_date.

3.1.3.3 It then decrements the available_copies count in the Books table.

3.1.3.4 If available_copies is 0, it returns an error message.

4 **ReturnBook(p_transaction_id, p_return_date) (Procedure):**

4.1 **Purpose:** Automates the process of returning a book.

4.2 **Logic:**

4.2.1 It retrieves the book_id and due_date for the given p_transaction_id.

4.2.2 It calculates the fine by comparing p_return_date and v_due_date (assuming a fine of ₹10 per day).

4.2.3 It updates the Transactions record, setting the return_date, calculated fine_amount, and status to 'Returned'.

4.2.4 It increments the available_copies in the Books table.

5 **CalculateTotalFine(p_member_id) (Function):**

5.1 **Purpose:** Calculates the total outstanding (or paid) fine for a specific member.

5.2 **Logic:** It returns the sum of fine_amount from all transactions associated with the p_member_id. COALESCE is used to ensure it returns 0 instead of NULL if the member has no fines.

3.3. Views

Views are saved queries that act as virtual tables, simplifying complex data retrieval for reporting.

3.3.1 **Current Issued Books:**

3.3.1.1 **Purpose:** Provides a quick look at all books that are currently checked out (i.e., return_date is NULL).

3.3.1.2 **Details:** It joins Transactions, Members, and Books to show the member's name, book title, issue date, and due date.

3.3.2 **Library Stats:**

3.3.2.1 **Purpose:** Creates an "at-a-glance" dashboard of the entire library.

3.3.2.2 **Details:** It calculates and displays the total number of unique book titles, total copies of all books, total available copies, number of active members, and the count of currently issued books.

3.4. Sample Data

To test the system, sample data was inserted into the Members and Books tables. Several Transactions were also added to simulate a real-world scenario, including books that were returned on time, returned late (with fines), and some that are currently overdue.

4. Testing and Demonstration

The following queries were executed against the sample data to test the system's functionality.

4.1 Test Case 1: View Available Books

4.1.1 Query: SELECT book_id, title, author, available_copies FROM Books WHERE available_copies > 0 ORDER BY title;

4.1.2 **Purpose:** To ensure the available_copies count is being correctly managed by the IssueBook and ReturnBook procedures.

4.1.3 **Expected Result:** A list of all books with 1 or more available copies.

4.2 Test Case 2: View Overdue Books

4.2.1 **Query:** (From the "Demonstration Queries" section)

```
SELECT
    t.transaction_id, m.full_name, b.title, t.due_date,
    DATEDIFF(CURDATE(), t.due_date) AS days_overdue
FROM Transactions t
JOIN Members m ON t.member_id = m.member_id
JOIN Books b ON t.book_id = b.book_id
WHERE t.return_date IS NULL AND t.due_date < CURDATE();
```

4.2.2 **Purpose:** To identify all members with books that are currently overdue.

4.2.3 **Result (Based on Sample Data):** This query correctly identifies transaction ID 2 (Priya Singh, 'Wings of Fire') as overdue.

4.3 Test Case 3: Library Statistics

4.3.1 Query: SELECT * FROM LibraryStats;

4.3.2 Purpose: To test the LibraryStats view.

4.3.3 Result (Based on Sample Data): The view correctly aggregates all data, showing 6 unique book titles, 30 total copies, 23 available copies, 4 active members, and 3 currently issued books.

4.4 Test Case 4: Members with Fines

4.4.1 Query: (From the "Demonstration Queries" section)

```
SELECT m.member_id, m.full_name, SUM(t.fine_amount) AS total_fine
FROM Members m
JOIN Transactions t ON m.member_id = t.member_id
WHERE t.fine_amount > 0
GROUP BY m.member_id, m.full_name;
```

4.4.2 Purpose: To list all members who have an outstanding fine balance.

4.4.3 Result (Based on Sample Data): This query correctly lists Priya Singh (₹50.00) and Neha Gupta (₹25.00).

4.5 Test Case 5: Issuing a Book (Procedure)

4.5.1 **Query:** CALL IssueBook(2, 3, CURDATE()); (Priya Singh attempts to borrow 'The God of Small Things')

4.5.2 **Purpose:** To test the IssueBook stored procedure.

4.5.3 **Result:** The procedure executes successfully. available_copies for book_id 3 is decremented from 4 to 3, and a new record is added to Transactions.

4.6 Test Case 6: Calculating a Member's Fine (Function)

4.6.1 **Query:** SELECT CalculateTotalFine(1) AS total_fine_for_rahul;

4.6.2 **Purpose:** To test the CalculateTotalFine function.

4.6.3 **Result:** The query returns 0.00, which is correct as Rahul's sample transaction (ID 1) was returned on time.

5 Innovativeness & Usefulness

Innovativeness: While a library management system is a common project, the innovation lies in creating a lightweight, purely SQL-based backend. By embedding all business logic into stored procedures (IssueBook, ReturnBook) and functions (CalculateTotalFine), the database itself becomes the "application." This provides a highly efficient, secure, and data-centric solution that is independent of any specific front-end technology.

Usefulness: This project offers significant practical benefits for any small institution:

5.1 **Automation:** Eliminates hours of manual work for librarians, freeing them for other tasks.

5.2 **Accuracy:** Greatly reduces human error in tracking due dates and fines.

5.3 **Data-Driven Decisions:** The LibraryStats view allows librarians to instantly see which books are most popular, how many books are out, and how many members are active, enabling better purchasing and management decisions.

5.4 **Accountability:** Provides a clear, auditable trail of all transactions, including member fines.

6. Current Status of Development

The project is at the "functional backend" stage.

6.1 **Database Design:** The schema, tables, and relationships are finalized and implemented.

6.2 **Core Logic:** The stored procedures and functions for all primary operations (issuing, returning, fine calculation) are complete and tested.

6.3 **Reporting:** Basic reporting capabilities are in place via SQL views.

6.4 **Testing:** The backend has been populated with sample data and tested with demonstration queries.

The primary missing component is a user-facing application (front-end).

7. Market Potential & Competitive Advantage

Market Potential: The target market for this system includes:

7.1 Small community libraries.

7.2 School libraries (primary or secondary).

7.3 University department libraries.

7.4 Corporate or office libraries.

8 **Individuals with large personal collections.**

These organizations often cannot afford expensive, large-scale commercial library management software.

Competitive Advantage:

8.1 **Zero Cost:** As the system is built on open-source SQL databases (like MySQL), there are no licensing or purchasing costs, unlike commercial competitors.

8.2 **Customizable:** The SQL code is open and can be easily modified to fit specific needs (e.g., changing the fine amount, loan duration, or adding new fields).

8.3 **Lightweight:** The system requires minimal resources, running efficiently on basic server hardware.

8.4 **Platform-Independent Backend:** The database can be accessed by any front-end technology (Web, Desktop, Mobile), giving an organization complete flexibility.

8. Conclusion and Future Scope

8.1. Conclusion

This project successfully demonstrates the design and implementation of a relational database for a Library Management System. By utilizing SQL, the system provides a robust, scalable, and efficient solution for managing books, members, and transactions. The use of stored procedures, functions, and views encapsulates complex logic, ensures data integrity, and simplifies reporting. The system effectively solves the problems of manual tracking and provides a solid foundation for a complete library automation tool.

8.2. Future Scope [This section corresponds to "Anything else..." from the synopsis]

This backend-only project has significant potential for expansion. Future development efforts could include:

8.2.1 Front-End Development:

- **Web Application:** Building a web-based interface (e.g., using PHP, Python with Django/Flask, or Java with Spring Boot) would make the system accessible to librarians and members through a browser.
- **Desktop Application:** A desktop GUI (e.g., using Java Swing, .NET C#) could be built for high-performance use by library staff.

8.2.2 User Authentication & Roles: Implementing a Users table with roles (e.g., 'Admin', 'Librarian', 'Member') to control access and permissions.

8.2.3 Book Reservation System: Adding functionality for members to reserve books that are currently checked out.

8.2.4 Barcode Integration: Modifying the front-end to allow for scanning book ISBNs and member IDs via a barcode scanner to speed up the check-in/checkout process.

8.2.5 Enhanced Reporting: Creating more detailed reports, such as book borrowing history, most active members, and monthly fine collections.

9. Appendix: Complete SQL Code

```
-- Library Management System - Complete SQL Code
CREATE DATABASE IF NOT EXISTS library_management_system;
```

```sql
USE library_management_system;
-- Members Table
CREATE TABLE Members (
    member_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE,
    phone VARCHAR(15),
    membership_date DATE NOT NULL,
    status ENUM('Active', 'Inactive') DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Books Table
CREATE TABLE Books (
    book_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    author VARCHAR(100) NOT NULL,
    genre VARCHAR(50),
    published_year YEAR,
    isbn VARCHAR(20) UNIQUE,
    total_copies INT DEFAULT 1,
    available_copies INT DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- Transactions Table
CREATE TABLE Transactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    book_id INT NOT NULL,
    member_id INT NOT NULL,
    issue_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE NULL,
    fine_amount DECIMAL(8,2) DEFAULT 0.00,
    status ENUM('Issued', 'Returned', 'Overdue') DEFAULT 'Issued',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
```

-- Insert Sample Members

INSERT INTO Members (Full_name, Email_id, Phone_no, Membership_date, status) VALUES
('Aviral verma', 'aviralverma4850@gmail.com', '8303283566', '2025-01-01', 'Active'),
('Ashwani Tripathi', 'ashwanitripathi586@email.com', '9696901760', '2025-02-01', 'Active'),
('Ashutosh Mishra', 'ashutoshmishra878@gmail.com', '6307006620', '2025-02-15', 'Active'),
('Udit Raj pandey', 'uditrajpandey82266@gmail.com', '6393599233', '2024-03-05', 'Inactive'),
('Ritesh pandey', 'riteshpandey5694@gmail.com', '8858946573', '2024-02-28', 'Active');

-- Insert Sample Books

INSERT INTO Books (Title, Author_name, Genre, Published_year, isbn, Total_copies, Available_copies) VALUES

('The Palace of Illusions', 'Chitra Banerjee Divakaruni', 'Mythological Fiction', 2008, '9780385515998', 10, 8),
('Immortals of Meluha', 'Amish Tripathi', 'Mythology', 2010, '9789380658742', 14, 10),

('Five Point Someone', 'Chetan Bhagat', 'Contemporary Fiction', 2004, '9788129135513', 12, 9),

('The Secret of the Nagas', 'Amish Tripathi', 'Fantasy', 2011, '9789381626344', 9, 6),

('The Guide', 'R. K. Narayan', 'Fiction', 1958, '9788185986005', 7, 5),

('Midnight’s Children', 'Salman Rushdie', 'Historical Fiction', 1981, '9780099578512', 11, 7);

-- Insert

Sample Transactions

INSERT INTO Transactions (book_id, member_id, issue_date, due_date, return_date, fine_amount, status) VALUES

(1, 1, '2024-03-01', '2024-03-15', '2024-03-14', 0.00, 'Returned'),
(2, 2, '2024-03-05', '2024-03-19', NULL, 50.00, 'Overdue'),
(3, 3, '2024-03-10', '2024-03-24', NULL, 0.00, 'Issued'),
(4, 1, '2024-03-12', '2024-03-26', NULL, 0.00, 'Issued'),
(1, 4, '2024-02-20', '2024-03-05', '2024-03-10', 25.00, 'Returned');

-- Procedure to Issue a Book

DELIMITER $$

CREATE PROCEDURE IssueBook(

    IN p_member_id INT,

    IN p_book_id INT,

    IN p_issue_date DATE

)

```
BEGIN
    DECLARE available_count INT;
    DECLARE due_date DATE;
    SELECT available_copies INTO available_count
    FROM Books WHERE book_id = p_book_id;

    IF available_count > 0 THEN
        SET due_date = DATE_ADD(p_issue_date, INTERVAL 14 DAY);
        INSERT INTO Transactions (book_id, member_id, issue_date, due_date, status)
        VALUES (p_book_id, p_member_id, p_issue_date, due_date, 'Issued');
        UPDATE Books
        SET available_copies = available_copies - 1
        WHERE book_id = p_book_id;
        SELECT 'Book issued successfully' AS message;
    ELSE
        SELECT 'Book not available' AS message;
    END IF;
END$$
DELIMITER ;
-- Procedure to Return a Book
DELIMITER $$
CREATE PROCEDURE ReturnBook(
    IN p_Member_id INT,
    IN p_return_date DATE
)
BEGIN
    DECLARE v_book_id INT;
    DECLARE v_due_date DATE;
    DECLARE v_fine_amount DECIMAL(8,2);
    SELECT book_id, due_date INTO v_book_id, v_due_date
    FROM Transactions WHERE transaction_id = p_transaction_id;
    IF p_return_date > v_due_date THEN
        SET v_fine_amount = DATEDIFF(p_return_date, v_due_date) * 10;
    ELSE
        SET v_fine_amount = 0;
    END IF;
    UPDATE Transactions
```

```
    SET return_date = p_return_date,
        fine_amount = v_fine_amount,
        status = 'Returned'
    WHERE transaction_id = p_transaction_id;
    UPDATE Books
    SET available_copies = available_copies + 1
    WHERE book_id = v_book_id;
    SELECT CONCAT('Book returned successfully. Fine: ₹', v_fine_amount) AS message;
END$$
DELIMITER ;
-- Function to Calculate Total Fine for a Member
DELIMITER $$
CREATE FUNCTION CalculateTotalFine(p_member_id INT)
RETURNS DECIMAL(8,2)
READS SQL DATA
DETERMINISTIC
BEGIN
    DECLARE total_fine DECIMAL(8,2);
    SELECT COALESCE(SUM(fine_amount), 0) INTO total_fine
    FROM Transactions
    WHERE member_id = p_member_id AND fine_amount > 0;
    RETURN total_fine;
END$$
DELIMITER ;

-- Views
CREATE VIEW CurrentIssuedBooks AS
SELECT
    t.transaction_id,
    m.full_name AS member_name,
    b.title AS book_title,
    t.issue_date,
    t.due_date,
    t.status
FROM Transactions t
JOIN Members m ON t.member_id = m.member_id
JOIN Books b ON t.book_id = b.book_id
```

```
WHERE t.return_date IS NULL AND t.status != 'Returned';
CREATE VIEW LibraryStats AS
SELECT
    COUNT(*) AS total_books,
    SUM(total_copies) AS total_copies,
    SUM(available_copies) AS available_copies,
    (SELECT COUNT(*) FROM Members WHERE status = 'Active') AS active_members,
    (SELECT COUNT(*) FROM Transactions WHERE return_date IS NULL) AS
currently_issued_books
FROM Books;
-- Demonstration Queries
SELECT '=== AVAILABLE BOOKS ===' AS info;
SELECT Book_id, Title, Author_name, Available_copies
FROM Books WHERE available_copies > 0 ORDER BY title;
SELECT '=== TODAYS DUE BOOKS ===' AS info;
SELECT
    t.transaction_id,
    m.full_name,
    b.title,
    t.due_date
FROM Transactions t
JOIN Members m ON t.member_id = m.member_id
JOIN Books b ON t.book_id = b.book_id
WHERE t.return_date IS NULL AND t.due_date = CURDATE();
SELECT '=== OVERDUE BOOKS ===' AS info;
SELECT
    t.transaction_id,
    m.full_name,
    b.title,
    t.due_date,
    DATEDIFF(CURDATE(), t.due_date) AS days_overdue,
    (DATEDIFF(CURDATE(), t.due_date) * 10) AS calculated_fine
FROM Transactions t
JOIN Members m ON t.member_id = m.member_id
JOIN Books b ON t.book_id = b.book_id
WHERE t.return_date IS NULL AND t.due_date < CURDATE();
SELECT '=== LIBRARY STATISTICS ===' AS info;
```

```
SELECT * FROM LibraryStats;
SELECT '=== MEMBERS WITH PENDING FINES ===' AS info;
SELECT
    m.member_id,
    m.full_name,
    SUM(t.fine_amount) AS total_fine
FROM Members m
JOIN Transactions t ON m.member_id = t.member_id
WHERE t.fine_amount > 0
GROUP BY m.member_id, m.full_name
HAVING total_fine > 0;
-- Test Stored Procedures
SELECT '=== TESTING STORED PROCEDURES ===' AS info;
CALL IssueBook(2, 3, CURDATE());
SELECT CalculateTotalFine(1) AS total_fine;
```

Output:-

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Book_id	Title	Author_name	Available_copies
3	Five Point Someone	Chetan Bhagat	8
2	Immortals of Meluha	Amish Tripathi	10
6	Midnight's Children	Salman Rushdie	7
5	The Guide	R. K. Narayan	5
1	The Palace of Illusions	Chitra Banerjee Divakaruni	8
4	The Secret of the Nagas	Amish Tripathi	6
NULL	NULL	NULL	NULL

Books 33

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Member_id	Full_name	Email_id	Phone_no	Membership_date	status	created_at
1	Aviral verma	aviralverma4850@gmail.com	8303283566	2025-01-01	Active	2025-11-17 11:17:00
2	Ashwani Tripathi	ashwanitripathi586@email.com	9696901760	2025-02-01	Active	2025-11-17 11:17:00
3	Ashutosh Mishra	ashutoshmishra878@gmail.com	6307006620	2025-02-15	Active	2025-11-17 11:17:00
4	Udit Raj pandey	uditrajpandey82266@gmail.com	6393599233	2024-03-05	Inactive	2025-11-17 11:17:00
5	Ritesh pandey	riteshpandey5694@gmail.com	8858946573	2024-02-28	Active	2025-11-17 11:17:00
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Members 34

Commond:-

```
SELECT *
FROM Transactions;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Transaction_id	Book_id	Member_id	Issue_date	Due_date	Return_date	Fine_amount	status	Created_at
1	1	1	2024-03-01	2024-03-15	2024-03-14	0.00	Returned	2025-11-17 11:20:01
2	2	2	2024-03-05	2024-03-19	NULL	50.00	Overdue	2025-11-17 11:20:01
3	3	3	2024-03-10	2024-03-24	NULL	0.00	Issued	2025-11-17 11:20:01
4	4	1	2024-03-12	2024-03-26	NULL	0.00	Issued	2025-11-17 11:20:01
5	1	4	2024-02-20	2024-03-05	2024-03-10	25.00	Returned	2025-11-17 11:20:01
6	3	2	2025-11-17	2025-12-01	NULL	0.00	Issued	2025-11-17 12:02:45
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Command :- SELECT
```
    t.Transaction_id,
    m.Full_name AS member_name,
    b.Title AS book_title,
    t.Issue_date,
    t.Due_date,
    t.status
FROM Transactions t
JOIN Members m ON t.member_id = m.Member_id
JOIN Books b ON t.book_id = b.Book_id
ORDER BY t.Transaction_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

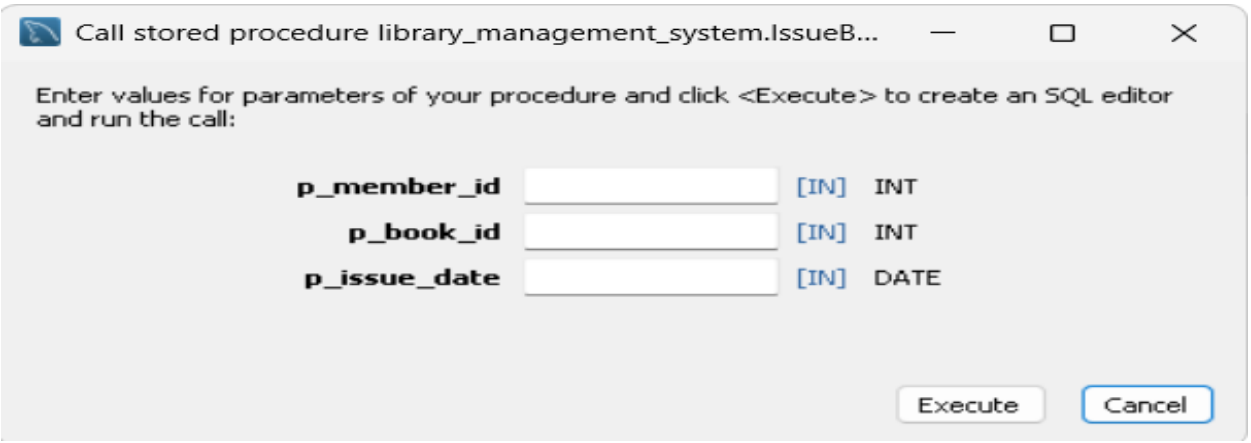
Transaction_id	member_name	book_title	Issue_date	Due_date	status
1	Aviral verma	The Palace of Illusions	2024-03-01	2024-03-15	Returned
2	Ashwani Tripathi	Immortals of Meluha	2024-03-05	2024-03-19	Overdue
3	Ashutosh Mishra	Five Point Someone	2024-03-10	2024-03-24	Issued
4	Aviral verma	The Secret of the Nagas	2024-03-12	2024-03-26	Issued
5	Udit Raj pandey	The Palace of Illusions	2024-02-20	2024-03-05	Returned
6	Ashwani Tripathi	Five Point Someone	2025-11-17	2025-12-01	Issued

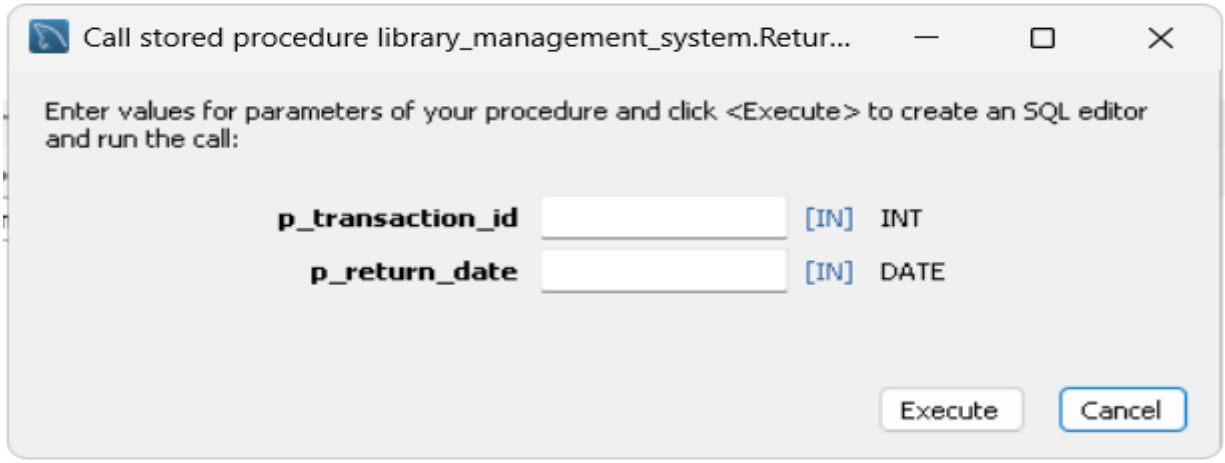
View already Created show Simple command is:-

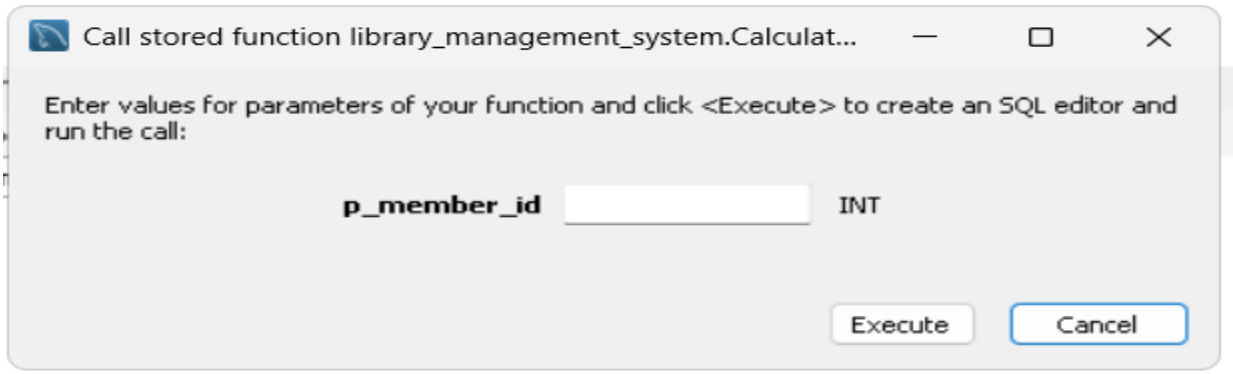
SELECT * FROM LibraryStats;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Total_books	Total_copies	Available_copies	Active_members	currently_issued_books
6	63	44	4	4







10. References

1. Elmasri, R., & Navathe, S. B. (2019). Fundamentals of Database Systems (7th ed.). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.
3. Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.
4. Beaulieu, A. (2020). Learning SQL: Generate, Manipulate, and Retrieve Data (3rd ed.). O'Reilly Media.
5. DuBois, P. (2014). MySQL Cookbook: Solutions for Database Developers and Administrators (3rd ed.). O'Reilly Media.

6. Oracle Corporation. (2025). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/
7. Chen, P. P. (1976). The Entity-Relationship Model: Toward a Unified View of Data. ACM Transactions on Database Systems, 1(1), 9–36.
8. Satzinger, J. W., Jackson, R. B., & Burd, S. D. (2016). Systems Analysis and Design in a Changing World (7th ed.). Cengage Learning.
9. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
10. Welling, L., & Thomson, L. (2017). PHP and MySQL Web Development (5th ed.). Addison-Wesley Professional. (Relevant for Future Scope 8.2.1)
11. Schildt, H. (2018). Java: The Complete Reference (11th ed.). McGraw-Hill Education. (Relevant for Future Scope 8.2.1)
12. Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media. (Relevant for Future Scope 8.2.1)
13. Kaur, H., & Verma, S. (2020). Design and Implementation of a Web-Based Library Management System. International Journal of Computer Applications, 175(12), 40-44.